

PEC3 - Tercera prueba de evaluación continua

Presentación

Esta PEC se focaliza en los circuitos secuenciales. Los circuitos combinacionales nos permiten describir funcionalidades de un circuito pero no nos permiten guardar información. Mediante biestables y registros podemos guardar información en memoria y hacer circuitos más complejos. En esta PEC practicaremos con este tipo de circuitos.

Competencias

- Entender el funcionamiento de los circuitos lógicos secuenciales, conocer y saber aplicar técnicas de diseño de sistemas secuenciales.

Objetivos

- Saber discernir, a partir de la funcionalidad que se quiere que tenga un circuito lógico, si el circuito debe de ser de tipo secuencial o combinacional.
- Conocer el funcionamiento del biestable D y de todas las entradas de control que puede tener.
- Saber analizar un circuito secuencial.
- Saber realizar un cronograma a partir de un circuito digital secuencial.
- Saber analizar un grafo de estados.
- Saber diseñar un circuito cualquiera a partir de la descripción de su funcionalidad mediante el modelo de Moore.

Recursos

Los recursos que se recomienda usar por esta PEC son los siguientes:

Básicos: El módulo 4 de los materiales.

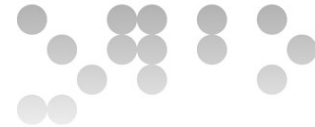
Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La puntuación está indicada en cada uno de los subapartados.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.

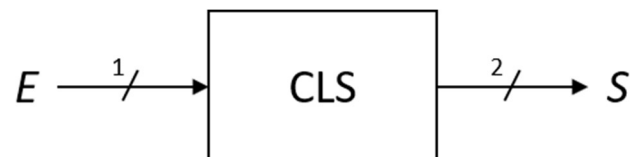


- Hay que entregar la solución en un fichero PDF usando una de las plantillas publicadas juntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de Entrega y registro de AC del apartado de Evaluación de vuestra aula.
- La fecha límite de entrega es el **25 de noviembre** (a las 24 horas).

Solución de la PEC

Ejercicio 1 [25 %]

Se quiere diseñar el grafo de estados de un circuito lógico secuencial (CLS) con una entrada E de un bit y una salida S de dos bits, tal y como muestra la figura siguiente:



El circuito tiene que leer los bits que entran por la señal E y tiene que detectar secuencias de unos, definidas como una tira de bits que empieza por 0, seguido de un cierto número de 1s, y acabada por un 0. Por ejemplo, se consideran secuencias de unos las siguientes secuencias: 010, 0111111110, 01110, etc. Se tiene que considerar que el último 0 de una secuencia puede ser, a su vez, el primero 0 de la secuencia siguiente.

Una vez detectada una secuencia de unos, en el ciclo siguiente la salida S del circuito valdrá el número de 1s que haya en la secuencia (con un máximo de 3). Es decir, si el número de 1s de la secuencia es menor o igual a 3 la salida será el número de 1s, y si el número de 1s es mayor a 3, entonces la salida será 3. En cualquiera otro caso el valor de la salida S será 00.

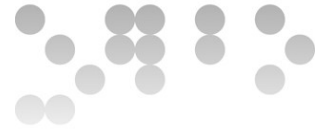
Ejemplo de funcionamiento:

Entrada E	0	0	1	0	1	1	1	0	...
Salida S	00	00	00	00	01	00	00	00	11

Se pide que diseñéis el grafo de estados de este circuito, especificando claramente la leyenda del mismo.

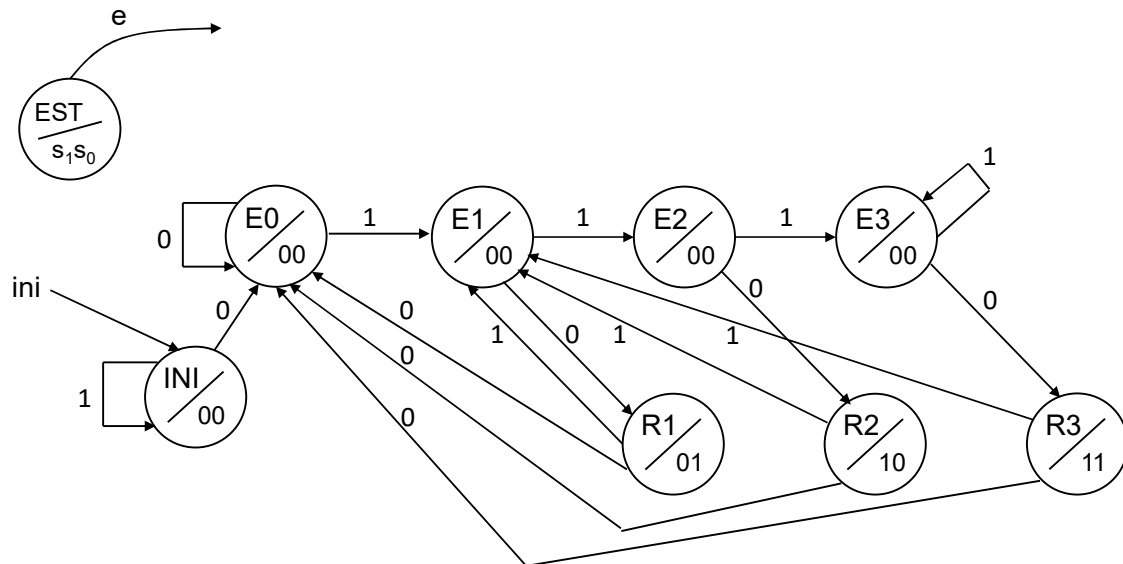
Para conseguir el funcionamiento deseado, el circuito tiene que tener los siguientes estados:

Estado	Descripción	Salida
INI	Estado inicial, no se ha reconocido nada de la secuencia	00
$E0$	Se ha visto uno o más 0	00
$E1$	Se ha visto el primer 1	00
$R1$	Se ha reconocido la secuencia 010	01
$E2$	Se ha visto el segundo 1	00
$R2$	Se ha reconocido la secuencia 0110	10



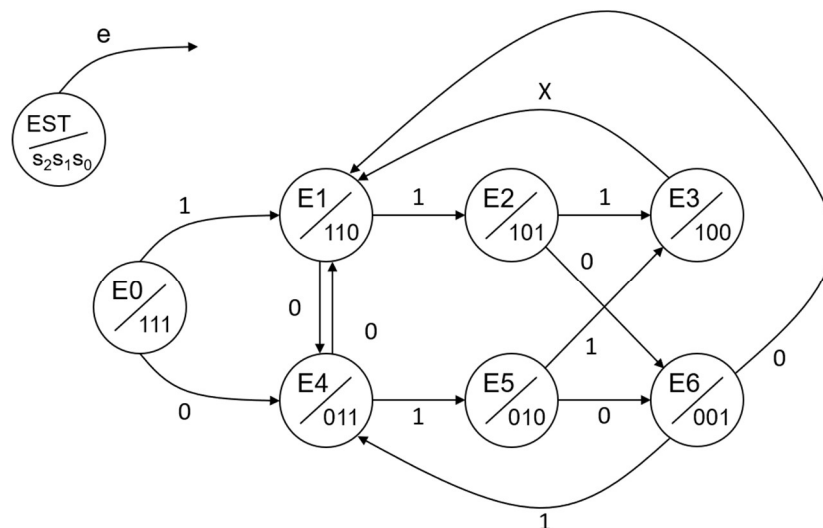
E3	Se han visto al menos tres 1	00
R3	Se ha reconocido una secuencia con al menos tres 1	11

El siguiente grafo representa el comportamiento deseado:



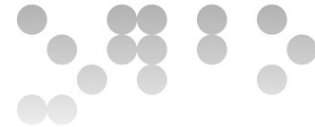
Ejercicio 2 [25 %]

Dado el grafo de estados siguiente:



Se pide:

- a) **[5 %]** ¿Cuántos bits de entrada tiene el circuito que implementa este grafo? ¿Cuántos bits de salida? ¿Cuál será el número mínimo de biestables para implementarlo? Si lo implementamos usando una memoria ROM, ¿cuántos bits de direcciones y cuántos bits de datos necesitará esta memoria?



El circuito tiene un bit de entrada, denominado e . Los bits de salida son tres, denominados $s_2 s_1 s_0$. Como el circuito tiene 7 estados necesitamos 3 bits para representarlos y, por lo tanto, necesitamos 3 biestables para poder almacenar estos 3 bits.

La memoria ROM que puede implementar este circuito tendría 4 bits de direccionamiento, 3 bits para el estado más 1 bit para la entrada. El ancho de las palabras de la ROM tendría que ser de 6 bits, 3 bits para guardar la salida y 3 bits para codificar el estado futuro.

- b) **[10 %]** Escribid la tabla de transiciones y la tabla de salidas del sistema representado por el grafo, codificando los estados según su índice asociado. Al escribir la tabla, poned en primer lugar las variables que codifican el estado y a continuación las variables de entrada.

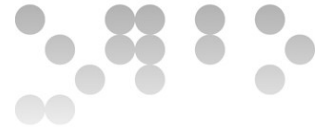
Nota: Tenéis disponible el ejercicio en VerilCHART. Fijaos que la tabla de transiciones y la de salidas se han unificado en una única tabla de excitaciones. Para poder probar este ejercicio en VerilCHART tenéis que reemplazar los bits *don't care* por valores 0.

La codificación de los estados según su índice se muestra en la siguiente tabla:

Codificación de los estados	
Estado	$q_2 q_1 q_0$
$E0$	000
$E1$	001
$E2$	010
$E3$	011
$E4$	100
$E5$	101
$E6$	110

La tabla de transiciones establece cuál es el estado en el ciclo siguiente a partir del valor que toman las señales de entrada.

$q_2 q_1 q_0$	e	$q_2^+ q_1^+ q_0^+$
000	0	100
000	1	001
001	0	100
001	1	010
010	0	110
010	1	011
011	0	001
011	1	001
100	0	001
100	1	101
101	0	110
101	1	011
110	0	001
110	1	100
111	0	XXX



111	1	XXX
-----	---	-----

La tabla de salidas determina el valor de las señales de salida para cada estado.

Estado	$s_2 s_1 s_0$
000	111
001	110
010	101
011	100
100	011
101	010
110	001

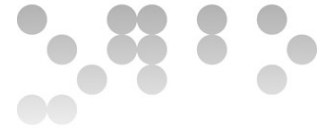
Finalmente, unificamos las dos tablas en una, teniendo en cuenta que las salidas (s_2 , s_1 , s_0) dependen del estado, no del estado+. A continuación, se muestra la tabla unificada:

$q_2 q_1 q_0$	e	$q_2 + q_1 + q_0 +$	$s_2 s_1 s_0$
000	0	100	111
000	1	001	111
001	0	100	110
001	1	010	110
010	0	110	101
010	1	011	101
011	0	001	100
011	1	001	100
100	0	001	011
100	1	101	011
101	0	110	010
101	1	011	010
110	0	001	001
110	1	100	001
111	0	XXX	XXX
111	1	XXX	XXX

- c) [10 %] Especificad en binario y en hexadecimal el contenido de todas las posiciones de la memoria ROM descrita al apartado anterior.

Nota: Tenéis disponible el ejercicio en VerilCIRC. Para poder probar este ejercicio en VerilCIRC tenéis que reemplazar los bits *don't care* por valores 0.

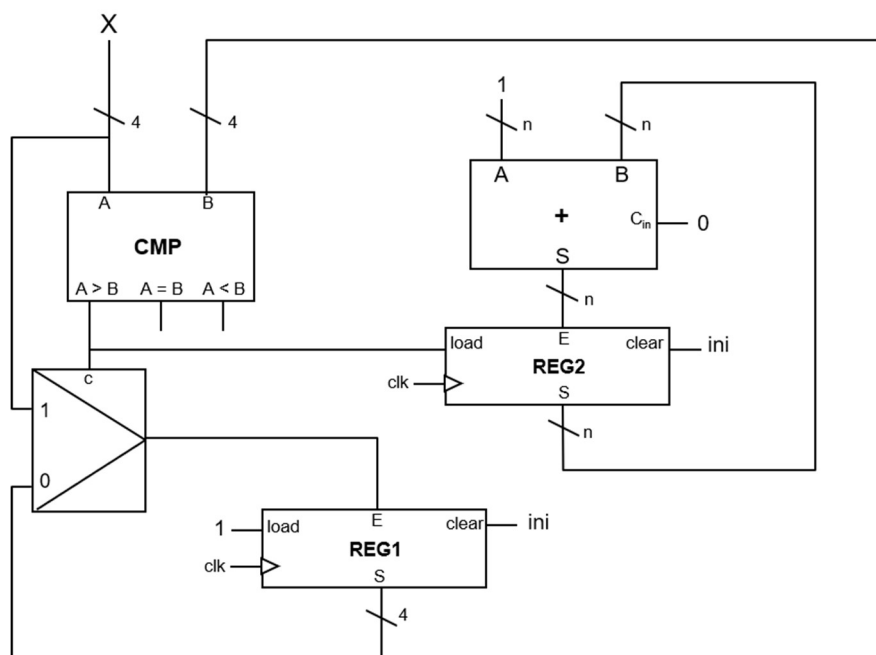
El direccionamiento de la memoria ROM viene definido por las señales $q_2 q_1 q_0 e$, especificadas en orden de mayor a menor peso. Cada posición de la memoria almacena los bits $q_2 + q_1 + q_0 + s_2 s_1 s_0$ especificados en orden de mayor a menor peso. El contenido de la memoria ROM será:

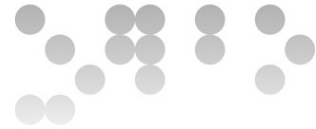


Dirección	Contenido en binario	Contenido en hexadecimal
0	100111	27
1	001111	0F
2	100110	26
3	010110	16
4	110101	35
5	011101	1D
6	001100	0C
7	001100	0C
8	001011	0B
9	101011	2B
10	110010	32
11	011010	1A
12	001001	09
13	100001	21
14	XXXXXX	XX
15	XXXXXX	XX

Ejercicio 3 [25 %]

Dado el circuito siguiente:





Suponed que inicialmente la señal *ini* vale 1 durante cierto rato, después pasa a 0 y ya no vuelve a cambiar de valor.

Describid el comportamiento del circuito a partir del momento en que *ini* pasa a 0, respondiendo las preguntas siguientes. Responded sólo con palabras (no hace falta que uséis expresiones booleanas ni tablas).

a) [10 %] ¿Cómo progresarán los valores del registro *REG1*?

Con la señal *load* = 1, *REG1* se cargará en cada flanco ascendente con el valor más grande entre su valor (salida *REG1*) y *X*, determinado por el comparador.

b) [10 %] ¿Cómo progresarán los valores del registro *REG2*?

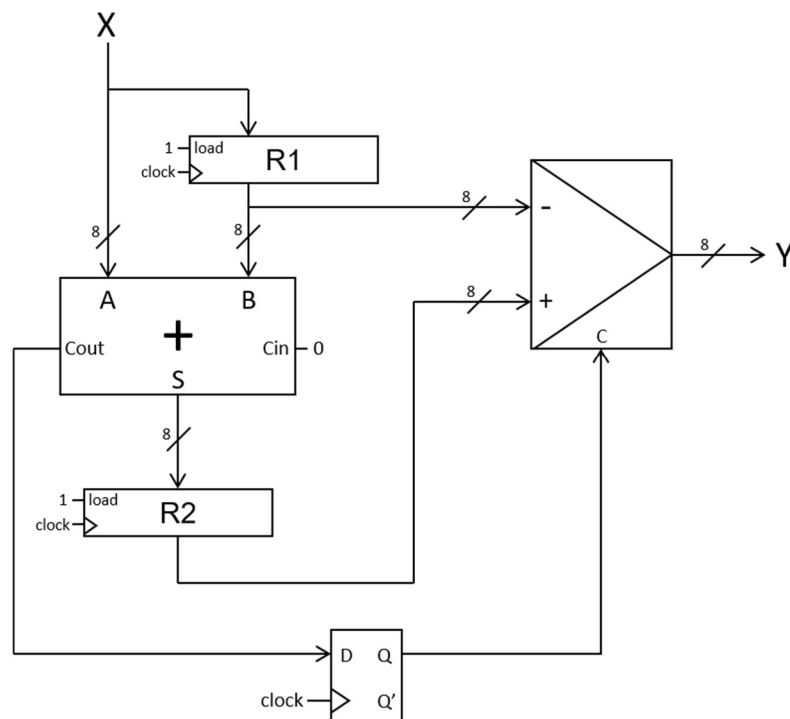
El valor de *REG2* se incrementará en 1 en cada flanco ascendente en caso de que la salida *A > B* del comparador sea 1, es decir, cuando el valor de *X* sea más grande que el valor de *REG1*. Es decir, *REG2* guarda el número de ocasiones en que $X > REG1$.

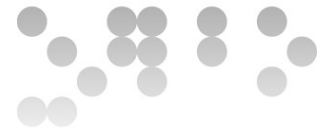
c) [5 %] ¿Cuál es el valor más pequeño para *n* tal que para cualquier secuencia de valores de *X* no se produzca desbordamiento en *REG2*?

REG1 se inicia a 0. El valor más grande que puede tomar *X* con 4 bits es 15. Si el valor de *X* es 1 en el primer ciclo y se incrementa cada ciclo en 1, la entrada *X* habrá sido más grande que *REG1* 15 veces. Con ninguna otra secuencia de valores de *X* ocurrirá que *X* haya sido más grande que *REG1* más veces. Por lo tanto, el valor máximo que puede tomar *REG2* es 15. Para representar el valor 15, *REG2* necesita 4 bits y *n* tiene que valer 4.

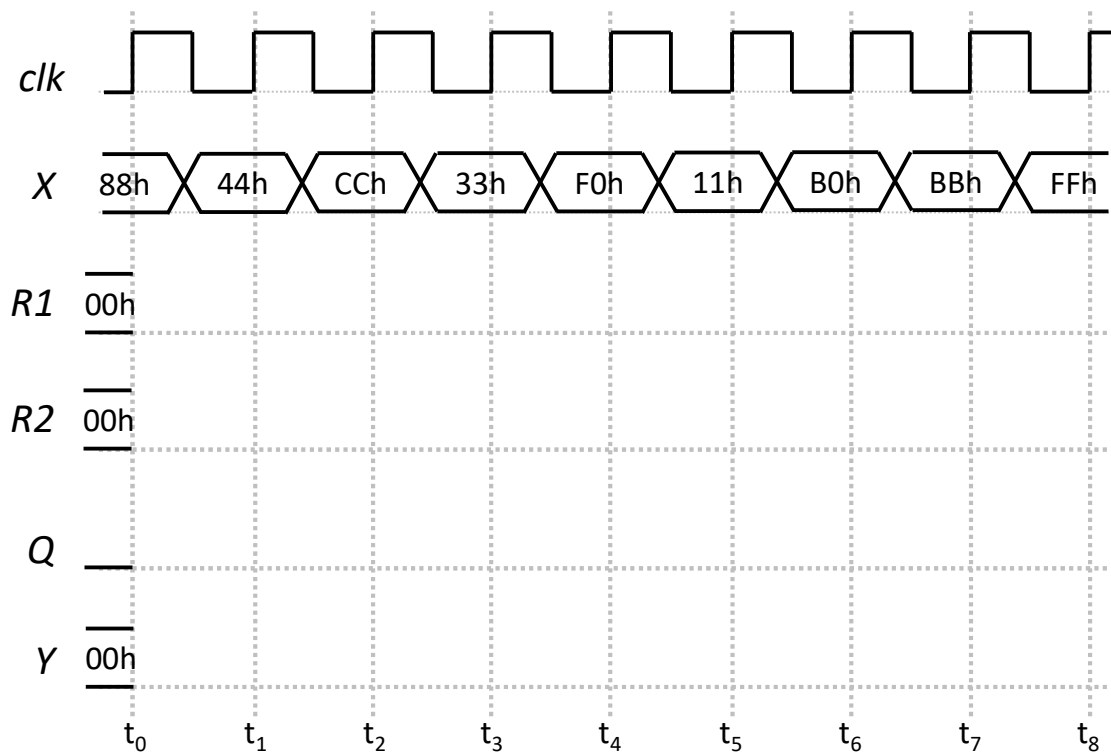
Ejercicio 4 [25 %]

Dado el circuito siguiente:





Completad el cronograma siguiente, poniendo los valores de los registros en hexadecimal.



Nota: Tenéis disponible este ejercicio en VerilCHART.

El registro *R1* tiene la señal *load*=1 de forma permanente. Como consecuencia, en cada flanco ascendente de la señal de reloj el registro *R1* carga el valor *X* que está en su entrada.

El registro *R2* también tiene la señal *load*=1 de forma permanente. Por eso en cada flanco ascendente de la señal de reloj el registro *R2* carga el valor que tiene la salida del sumador. Este valor corresponde a la suma de la señal *X* con la salida del registro *R1*.

Señal *Q* y salida *Y*: La suma que realiza el sumador entre los instantes t_1 y t_2 , t_2 y t_4 , t_4 y t_5 produce que se active la señal *C_{out}* a 1 y *D*=1 de la entrada del biestable. En el próximo flanco ascendente de la señal de reloj la salida *Q* del biestable se activa a 1 y por lo tanto la entrada de control del multiplexor tiene el valor 1. Por lo tanto, durante este ciclo, el valor de *R2* aparece en la salida *Y* del multiplexor. Mientras *Q*=0, la salida *Y* del multiplexor corresponde al valor del registro *R1*.

